

Task 3

API SECURITY RISK ANALYSIS REPORT

Name: Bhavyasri J

Internship: Cyber Security Internship –Future Interns

Task: Task 3 – API Security Risk Analysis

Date: [Current Date]

1. Introduction

Application Programming Interfaces (APIs) are widely used in modern web applications, mobile applications, and Software-as-a-Service (SaaS) platforms. Improperly secured APIs can lead to unauthorized access, excessive data exposure, abuse of endpoints, and service disruption.

This report presents a read-only security assessment of a public API with the objective of identifying common API security risks and providing recommendations based on industry best practices.

2. Objective

The objective of this assessment is to:

- Analyze a public API in a safe and ethical manner.
- Inspect responses and endpoint behavior.
- Identify potential API security risks.
- Assess their severity and business impact.
- Recommend suitable mitigation measures.

3. Scope of Assessment

Allowed Activities

- Documentation review
- Read-only GET requests
- Header and response inspection
- Security risk analysis

Restricted Activities

- Exploitation attempts
- Authentication bypass
- Denial-of-Service testing
- Attacks against private systems

4. Tools Used

Tool	Purpose
Postman	API testing and response analysis
Browser Developer Tools	Header inspection
GitHub	Project repository
MS Word / LibreOffice Writer Documentation	

5. Target API

API Name

JSONPlaceholder Test API

Base URL

<https://jsonplaceholder.typicode.com>

Endpoints Tested

1. GET /posts

<https://jsonplaceholder.typicode.com/posts>

2. GET /users

<https://jsonplaceholder.typicode.com/users>

3. GET /comments

<https://jsonplaceholder.typicode.com/comments>

6. Methodology

The assessment was performed using the following process:

1. Selected the JSONPlaceholder public API.
2. Reviewed the API documentation.
3. Sent GET requests through Postman.
4. Observed headers and response bodies.
5. Identified potential security risks.
6. Classified the risks according to severity.
7. Proposed remediation recommendations.

7. Security Findings

Finding 1: Missing Authentication

Severity

Medium

Observation

The tested endpoints can be accessed without authentication mechanisms such as API keys or tokens.

Business Impact

If sensitive APIs are exposed in production environments, attackers may gain unauthorized access to valuable information.

Recommendation

Implement:

- API Keys
- JWT Authentication
- OAuth 2.0

Finding 2: Excessive Data Exposure

Severity

Medium

Observation

The "/users" endpoint exposes user-related information including:

- Email addresses
- Phone numbers
- Website information
- Address details

Business Impact

Attackers can use this information for phishing attacks and social engineering campaigns.

Recommendation

Expose only required data and avoid returning unnecessary information.

Finding 3: Missing Rate Limiting

Severity

High

Observation

No request limiting mechanisms were observed.

Business Impact

Attackers may abuse endpoints and overload servers, causing service degradation and denial-of-service conditions.

Recommendation

Implement:

- Rate limiting
- Request throttling
- API gateways

Finding 4: Input Validation Risks

Severity

Low

Observation

Improper input validation may allow malicious payloads to affect backend processing.

Business Impact

Poor validation practices can result in injection attacks and unexpected application behavior.

Recommendation

Validate and sanitize all incoming inputs before processing.

8. Risk Summary

Security Risk	Severity
Missing Authentication	Medium
Excessive Data Exposure	Medium
Missing Rate Limiting	High
Input Validation Issues	Low

9. Screenshots

Figure 1: GET /posts Response

(Insert Screenshot)

Figure 2: GET /users Response

(Insert Screenshot)

Figure 3: GET /comments Response

(Insert Screenshot)

10. Conclusion

The JSONPlaceholder API is designed for educational and testing purposes. The assessment demonstrated several common API security concerns including missing authentication, excessive data exposure, absence of rate limiting, and input validation issues.

Although these findings are acceptable for a test API, similar weaknesses in production environments could expose organizations to data breaches and service abuse. Implementing authentication mechanisms, minimizing exposed data, applying rate limiting, and validating inputs are essential best practices for securing modern APIs.

